

SAP DOCS · FIELD REFERENCE

T-Codes & Tables

How to reach any screen in SAP — and find the data behind it

"Every SAP developer question eventually becomes the same question: which table is this in? The people who answer it in five seconds and the people who lose an afternoon are separated by one thing — they learned the table names."

10

PARTS

35

TOPICS

40+

TABLES

Table of Contents

- T-Codes & Tables
- How to Use This Guide
- Part A — Why T-Codes Matter
 - A1. What a Transaction Code Really Is · A2. Why Fluency With T-Codes Reads as Experience · A3. How T-Codes Are Named · A4. Custom Z Transaction Codes
- Part B — The T-Codes That Show You Data
 - B1. SE11 — The ABAP Dictionary · B2. SE16N — The Data Browser · B3. SE80 — Seeing Everything in One Place · B4. SE93 and SE84 — Finding Things · B5. F1 → Technical Information — The Bridge
- Part C — Why Table Names Are the Real Vocabulary
 - C1. Why a Developer Must Know Tables · C2. The Naming Logic · C3. Three Ways to Find a Table You Do Not Know · C4. Keys, MANDT and How Tables Join
- Part D — Materials Management
 - D1. MARA and MAKT — The Material Master · D2. MARC, MARD, MBEW — Material Data by Level · D3. EBAN — Purchase Requisitions · D4. EKKO, EKPO, EKET, EKBE — Purchase Orders
- Part E — Sales and Distribution
 - E1. VBAK, VBAP, VBEP — Sales Orders · E2. LIKP and LIPS — Deliveries · E3. VBRK, VBRP and VBFA — Billing and Document Flow
- Part F — Finance
 - F1. BKPF and BSEG — The Classic Accounting Document · F2. ACDOCA — The Universal Journal · F3. SKA1, SKB1 and T001 — Accounts and Company Codes
- Part G — Partners, Inventory and Organisation
 - G1. KNA1, LFA1, BUT000 — Customers, Vendors, Business Partners · G2. MKPF, MSEG and MATDOC — Stock Movements · G3. The Organisational Tables
- Part H — The Other Modules, Briefly
 - H1. Production, Quality, Maintenance and Warehouse · H2. HR and the SuccessFactors Question
- Part I — System and Technical Tables
 - I1. TSTC, TADIR and the DD* Tables · I2. Transport and User Tables
- Part J — Putting It to Work
 - J1. From a Screen Field to a Working SELECT · J2. Views and CDS Equivalents in S/4HANA · J3. Practice Drills for Your 9–11 Slot · J4. Twenty Questions You Should Be Able to Answer

How to Use This Guide

Two vocabularies run SAP. **T-codes** are how you reach a screen. **Table names** are how you find the data behind it. Learn the first in a week; build the second over months. This guide gives you both, and — more importantly — the **method** for finding the ones it does not list.

The honest framing: there are around 100,000 tables in an S/4HANA system. Nobody memorises them. What experienced developers actually have is about **forty table names in permanent memory** plus a **reliable technique** for finding any of the rest in under a minute. This guide gives you the forty and the technique.

Telugu: SAP lo **rendu bhashalu (vocabularies)** unnayi. **T-code** — screen ki ela vellali. **Table peru** — aa screen venaka data ekkada undi. Modati di vaaram lo vastundi; rendodi nelalu pattutundi.

Nija vishayam: S/4HANA lo **lakshalaadi tables** unnayi — evaru anni gurthupettukoru. Experience unna developers daggara **naluvai (40) peru** gurthu untayi, inka **migilnavi nimisham lo vetike technique** untundi. Ee guide lo rendu unnayi.

How to work through it:

Part	What it covers	How to read it
A	Why T-codes matter, how they are named	Straight through — 20 minutes
B	The T-codes that show you data — SE11, SE16N, SE80, and the F1 trick	Straight through, then practise
C	Why table names are the real skill, and how to discover any table	The most important part
D–I	The tables themselves, grouped by business area	Reference — skim now, return often
J	Screen field → table → working SELECT, drills and interview questions	Do the drills in your 9–11 slot

If you only read two sections: read **C3** (three ways to find a table you do not know) and **J1** (the full walkthrough from a screen field to a running SELECT). Those two are the actual skill. Everything else is the raw material.

Part A — Why T-Codes Matter

A1. What a Transaction Code Really Is

Simple definition: a **transaction code** (T-code) is a short name — **SE11** , **MM03** , **VA03** — that starts a specific program or screen. Typing it into the command field takes you there instantly, from anywhere in SAP.

Telugu: T-code ante **chinna peru** — SE11, MM03, VA03. Command field lo type chesi Enter kotte, aa screen ki direct ga velthavu. SAP lo **anni panulu** T-codes dwaara ye jarugutayi. Web lo URL laantidi.

What is actually happening: a T-code is a **named entry point**. It is defined in transaction **SE93** and stored in table **TSTC**, and it records three things: the code, what it starts (a program, a screen, or a method), and the authorization required.

T-code	Starts	Type
SE38	Program SAPMS38M , screen 0100	Dialog transaction
SA38	A report by name	Report transaction

T-code	Starts	Type
VA03	Program + screen for displaying a sales order	Dialog transaction
ZNV_PR_LIST	Your own Z report	Report transaction

Anchor to what you know: a T-code is a **route** in an Express app. `app.get('/orders/:id', showOrder)` maps a short path to a handler. `VA03` maps a short code to a program. The command field is the address bar, and `/n` versus `/o` is "same tab" versus "new tab".

Why SAP works this way: SAP has tens of thousands of screens across every business function. A menu tree deep enough to reach them all would be unusable. Short codes flatten that entirely — `ME23N` takes the same time to reach whether it is two levels deep or nine.

The consequence for you: menus are for browsing; T-codes are for working. Nobody experienced navigates by menu.

A2. Why Fluency With T-Codes Reads as Experience

Simple definition: speed with T-codes is the fastest visible signal of whether someone actually uses SAP or has only studied it.

Telugu: Idi chinna vishayam la anipistundi, kaani **chala pedda signal**. Interview lo screen share chesi "table chudandi" ante — nuvvu vintane `/nse16n`` type cheste, adi nuvvu **roju vaadutunnavu** ani cheptundi. Menu lo vetikithe, "kevalam chadivadu" ani telisipotundi. Anduke **modati 15 T-codes** gattiga nerchuko.

Where it shows:

Situation	The fluent response	What hesitation signals
"Show me that table"	<code>/nse16n</code> , table name, F8 — five seconds	Studied, not used
"Check if it dumped"	<code>/nst22</code> immediately	Never handled an incident
"Where is that field stored?"	F1 → Technical Information	Would guess or search online
"Is the job finished?"	<code>/nsm37</code>	Has not worked a real system

This is not about memorising trivia. It is that the codes you know are the tasks you have actually done. An interviewer is not testing recall — they are inferring experience from reaction time.

The practical target: fifteen T-codes at automatic speed by December. They are in Part B and Part J4, and they are the ones you will use in your 9–11 slot anyway. Fluency arrives from use, not from flashcards.

One caution — do not fake it. Rattling off T-codes you have never opened collapses on the follow-up question ("what does the second tab do?"). Learn the fifteen properly rather than fifty superficially.

A3. How T-Codes Are Named

Simple definition: T-codes follow loose but real naming patterns. Knowing them lets you guess correctly and recognise unfamiliar codes.

Telugu: T-codes ki oka **pattern** undi. Adi telisthe, kotta code chusinappudu kuda adi e area di ani **oohinchagalavu**.
Udahanaki **SE** tho modalayithe developer tool, **SM** ante basis, **ST** ante monitoring.

The technical prefixes — your area:

Prefix	Area	Examples
SE	ABAP Workbench / development	SE11, SE24, SE37, SE38, SE80, SE93
SM	System management / basis	SM30, SM36, SM37, SM50, SM59
ST	System monitoring and trace	ST05, ST22, ST03
SU	Users and authorizations	SU01, SU3, SU53
SP	Spool and printing	SP01, SP02
S_	Various administrative	S_MEMORY_INSPECTOR

The business prefixes — modules:

Prefix	Module	Examples
MM	Material master	MM01, MM02, MM03
ME	Purchasing	ME21N, ME23N, ME51N
VA	Sales	VA01, VA02, VA03
VF	Billing	VF01, VF03
VL	Delivery	VL01N, VL03N
MB / MIGO	Inventory movements	MB51, MIGO
FB / FS	Finance	FB03, FS00
CO / CS	Production, BOM	CO01, CS03
IE / IW	Plant maintenance	IE03, IW31

The number pattern that repeats everywhere — learn this once:

Ending	Means
...01	Create
...02	Change
...03	Display

So MM01 creates a material, MM02 changes it, MM03 displays it. Same for VA01/02/03 , FK01/02/03 , XD01/02/03 . **When exploring an unfamiliar system, always use the 03 version** — display mode cannot break anything.

The N suffix means "new" (enjoy) transactions: ME21N replaced ME21 , VL01N replaced VL01 . They are single-screen redesigns from the ECC era. Always prefer the N version.

A4. Custom Z Transaction Codes

Simple definition: you can create your own T-code so users reach your program by a short name instead of running it from SE38.

Telugu: Nuvvu raasina program ki **sontha T-code** create cheyyochu — SE93 lo. Appudu users SE38 teruvakunda, direct ga ZNV_PR_LIST ani type chesi run cheyyochu. Nija projects lo prathi custom report ki oka Z T-code istaru.

How, in SE93:

1. /nSE93 → enter ZNV_PR_LIST → **Create**
2. Short text describing it
3. Choose the type — for a report, **"Program and selection screen (report transaction)"**
4. Enter the program name
5. Save into your package and transport request

Transaction type	Use for
Program and selection screen (report transaction)	A normal ABAP report — your default
Program and screen (dialog transaction)	A classical Dynpro program
Method of a class (OO transaction)	A class-based application
Transaction with parameters (parameter transaction)	Pre-filling another transaction's fields — e.g. an SM30 shortcut for one table

The parameter transaction is worth knowing. It creates a T-code that launches an existing transaction with fields pre-filled. The classic use: give users a code that opens SM30 directly on one specific table, so they never see the table-selection screen.

Why an ABAP developer cares: every custom report you build on a real project ends with "now give it a transaction code so the business can run it". It is the last step of the job, and forgetting it means users cannot reach your work.

Part B — The T-Codes That Show You Data

B1. SE11 — The ABAP Dictionary

Simple definition: SE11 shows you a table's **structure** — its fields, their types, its keys, and its relationships. It is where you go to understand *what a table is*.

Telugu: SE11 lo table yokka **nirmanam (structure)** chustavu — enni fields unnayi, e type, key fields evi. **Data chudavu** ikkada — data ki SE16N. SE11 = "ee table ela undi", SE16N = "ee table lo em undi". Ee teda gurthupettuko.

The distinction that confuses beginners:

	Shows you	Use it to
SE11	The structure — fields, types, keys	Understand what a table holds
SE16N	The contents — actual rows	See real data

What to do in SE11 when you meet a new table — this is a repeatable four-step routine:

1. `/nSE11` → *Database table* → type the name → **Display**
2. **Read the key fields first.** The fields with the Key flag define what makes a row unique. This tells you the *grain* of the table — is one row a purchase order, or one line of one purchase order?
3. **Scan the field list** for the data you need, and note the data elements.
4. **Check** `Goto → Foreign keys` or the *Entry help/check* tab to see which tables this one relates to.

The tabs, briefly:

Tab	Holds
Attributes	Delivery class, description, who created it
Delivery and Maintenance	Whether SM30 maintenance is allowed
Fields	The main event — every field, key flags, data elements, types
Entry help/check	Foreign keys and search helps
Currency/Quantity Fields	Which field holds the currency for each amount

The single most useful habit in this whole guide: every time you meet an unknown table name — in code, in a colleague's message, in an F1 popup — spend two minutes in SE11 Display reading its key fields. Do that daily and by December you will "just know" thirty tables without ever having sat down to memorise them.

B2. SE16N — The Data Browser

Simple definition: SE16N shows you the **rows** in any table. It is how you check what data actually exists.

Telugu: SE16N lo table lo unna **nijamaina rows** chustavu. "Naa SELECT lo data raaledu" ani anukunnappudu, modata SE16N lo chudu — table lo nijamga data unda leda ani. Chala sarlu problem code lo kaadu, **data lo** untundi.

The workflow that answers most questions:

Something is wrong with my report

SE16N on the table → Does the data even exist?

↓ yes

↓ no

Check my WHERE clause
and my JOIN

The problem is upstream,
not in my code

Features that make it fast — most people use none of these:

Feature	Where	Why
Multiple selection (the → arrow)	Beside any field	Several values, ranges, or <i>exclusions</i>
Wildcards * +	In any character field	PR* finds everything starting PR
Number of Entries	Toolbar	Counts rows without fetching them — safe on huge tables
Output Fields	Toolbar	Pick columns; a 200-field table becomes readable
Layouts	ALV toolbar	Save your column selection and reuse it
Export	List → Export → Spreadsheet	Get data into Excel

Always set a maximum hit count. On a table with fifty million rows, an unbounded selection produces a `TIME_OUT` dump and an unhappy basis team.

SE16 vs SE16N vs SE16H:

Note	
SE16	The classic Data Browser — present on every system, including ancient ones
SE16N	More flexible selection and output. Your default
SE16H	S/4HANA only — adds grouping, aggregation and totals, pushed down into HANA

On &SAP_EDIT : you will hear that typing it in SE16N enables direct table editing. It bypasses all validation, is logged and audited, and is disabled on most production systems. Do not use it, and never offer it as an answer in an interview. The correct answer is always *"through the transaction that owns the data, or a maintenance dialog"*.

B3. SE80 — Seeing Everything in One Place

Simple definition: SE80 is the Object Navigator — a tree of every object in a package, with an editor beside it.

Telugu: SE80 lo **package peru** ista chaalu — aa package lo unna anni tables, programs, classes tree lo kanapadutayi. Kotta project ki join ayinappudu, aa project ni ardham chesukovadaniki idi **best tool**.

Why it belongs in a guide about finding data: when you join a project and someone says "*our purchasing enhancements are in package ZMM_PROC*", SE80 with that package name shows you **every Z table, class, program and transaction** they built — in one tree. That is far faster than opening objects one at a time, and it is how you learn an unfamiliar codebase.

The object-type dropdown:

Choose	Type	You get
Package	ZMM_PROC	Everything in it, grouped by object type
Program	ZNV_PR_LIST	One program with its includes
Class/Interface	ZCL_PR_HANDLER	The class with its methods
Function Group	ZNV_FG01	The group and its function modules

The where-used list (Ctrl+Shift+F3) is the other reason SE80 matters. Before you change a table, a data element or a method, run it. If forty programs use that field, your "small change" is not small. Interviewers ask how you do impact analysis; this is the answer.

B4. SE93 and SE84 — Finding Things

Simple definition: the two search tools — SE93 for transaction codes, SE84 for everything else in the repository.

Telugu: Rendu vetike tools. **SE93** — T-codes vetakadaniki. **SE84** — repository lo **e object aina** vetakadaniki (tables, programs, classes — author batti, package batti, peru batti). SE84 chala takkuva mandi vaadutaru, kaani chala shakti vantidi.

SE93 — transaction codes:

- Type a code and display it to see exactly which program it starts
- Use **F4 with a wildcard** — enter **ME2*** and press F4 to list every purchasing display transaction
- Useful in reverse too: someone shows you a screen, you check the status bar for the T-code, then look it up here

SE84 — the Repository Information System:

Search the entire repository by criteria rather than by name:

Find	How
All tables starting ZMM*	Dictionary Objects → Database Tables → name ZMM*

Find	How
Everything a colleague created	Set <i>Created by</i> to their user ID
Everything in a package	Enter the package name
All programs changed last month	Set the change date range

A practical use for you right now: search SE84 for tables named **Z*** in your practice system. That shows you every custom table your institute or previous students created — real examples to open in SE11 and learn from.

B5. F1 → Technical Information — The Bridge

Simple definition: the trick that connects the two worlds. Press **F1** on any field on any SAP screen, then click **Technical Information**, and SAP tells you the **table and field name** storing that value.

Telugu: Idi ee guide lo **anni kanna mukhyamaina trick**. E screen lo, e field meeda aina **F1** kottu, tarvaata **Technical Information** click chey — aa value **e table lo, e field lo** undo SAP ye cheptundi. Table peru gurthupettukovadam avasaram ledu — **screen adigithe chaalu**.

Why this is the single most valuable page in this guide:

The everyday problem is: "*a functional consultant says the delivery date on the purchase order is wrong — write me a report.*" You do not know which table holds a PO delivery date. You have two options:

Approach	Result
Guess table names, search the internet	Ten minutes, possibly wrong
Open ME23N, click the field, F1 → Technical Information	Ten seconds, definitely right — EKET-EINDT

The exact steps:

1. Open the transaction in **display** mode (**ME23N** , **VA03** , **MM03**)
2. Click into the field you care about
3. Press **F1** — the help popup opens
4. Click **Technical Information** (a button in that popup)
5. Read **Table Name** and **Field Name**

What the popup shows you:

Row	Meaning
Table Name	The table — sometimes a <i>structure</i> , see the caution below
Field Name	The field in that table
Data Element	The data element, which you can open in SE11
Screen field	The technical name on the screen

One caution that saves confusion: sometimes the Table Name shown is a **structure** (often starting with `RM`, `MEPO`, or ending `_S`), not a real database table — because the screen is filled from a work area, not read directly. When that happens, take the **data element** name instead, open it in SE11, and use *Where-used* → *Table fields* to find the real tables that use it.

Practise it this week. Open ME23N or VA03, press F1 → Technical Information on five fields, and write the table names down. Do this for a week and you will have learned `EKKO`, `EKPO`, `EKET`, `VBAK`, `VBAP` without ever trying to memorise them — which is exactly how experienced developers actually learned them.

Part C — Why Table Names Are the Real Vocabulary

C1. Why a Developer Must Know Tables

Simple definition: in SAP, business logic is thin and data is enormous. Almost every task you are given is really a question about *where data lives*.

Telugu: SAP lo nee pani lo **chala varaku** "ee data ekkada undi?" ane prasnaki jawaabu vetakadam ye. Table perlu teliste nuvvu vegam; teliyakapote prathi chinna panini kuda gantalu teesukuntavu. Anduke **table perlu ye asalu vocabulary**.

Look at what a real week's tasks actually are:

The request	What it really means
"Build a report of open purchase orders"	Find the PO tables, know which field means "open"
"This total is wrong on the invoice"	Find the billing tables, trace the amount field
"Add the vendor's city to this list"	Know the vendor and address tables, and how they join
"Why did this material not get picked?"	Know stock and delivery tables

Not one of those is an algorithm problem. Every one is a **data-location** problem — which is exactly why the field suits you, and exactly why table knowledge is the skill that compounds.

How experienced developers actually know them: not from memorising a list. From meeting the same forty tables repeatedly, because SAP's core processes are the same everywhere. `EKKO` appears in every purchasing task at every company on earth. After the fifth time, it is permanent.

Your acceleration plan: the F1 → Technical Information habit from B5, plus two minutes in SE11 whenever you meet a new name. That converts months of passive exposure into weeks of deliberate learning.

C2. The Naming Logic

Simple definition: SAP table names look random but follow patterns from 1970s German abbreviations. You cannot always decode them — but the *structural* patterns are completely reliable.

Telugu: Table perlu German padala nunchi vachayi, anduke vintaga untayi. Perlu decode cheyyalemu, kaani **structure patterns** matram **eppudu okate**. Mukhyanga **header-item** pattern — idi telisthe sagam pani ayipoyinatte.

Pattern 1 — the header/item pair. This is the big one.

Almost every business document is stored in **two** tables: one row for the document, many rows for its lines.

Document	Header (1 row)	Item (many rows)
Purchase order	EKKO	EKPO
Sales order	VBAK	VBAP
Delivery	LIKP	LIPS
Billing document	VBRK	VBRP
Accounting document	BKPF	BSEG
Material document	MKPF	MSEG
Production order	AFKO	AFPO

Spot the sub-pattern: in SD, header tables end in **K** (*Kopf* = head) and item tables end in **P** (*Position* = item). Once you see **VBAK** / **VBAP** , **VBRK** / **VBRP** , **LIKP** / **LIPS** , the logic is obvious — and it lets you guess correctly.

Anchor to what you know: this is **orders** and **order_items** — the classic one-to-many you built in Phase 9. Identical modelling, older names. The join is always *header key* = *item key*, and the item table adds a line-number field.

Pattern 2 — configuration tables start with T.

Table	Holds
T001	Company codes
T001W	Plants
T001L	Storage locations
T024E	Purchasing organisations
TVKO	Sales organisations

If a table name starts with **T** followed by digits, it is almost certainly customizing — small, rarely changing, often buffered.

Pattern 3 — text tables end in T.

Descriptions live in separate tables, one row per language, because SAP is multilingual by design.

Data table	Text table	Extra key
MARA (material)	MAKT	SPRAS (language)

Data table	Text table	Extra key
T001W (plant)	T001W itself	—
TSTC (T-codes)	TSTCT	SPRAS
DD02L (tables)	DD02T	DDL LANGUAGE

The consequence you will hit: when your report shows a material number but no description, you forgot to join the text table **and filter on language** (**SPRAS** = sy-langu).

Pattern 4 — the module letter. Rough, but useful: **M*** material, **E*** purchasing (*Einkauf*), **V*** sales (*Vertrieb*), **L*** delivery/warehouse (*Lieferung/Lager*), **B*** accounting (*Buchhaltung*), **K*** customer/controlling, **A*** production orders, **P*** personnel.

C3. Three Ways to Find a Table You Do Not Know

Simple definition: the technique that replaces memorisation. Learn these three and no table is ever out of reach.

Telugu: Idi ee guide lo **anni kanna mukhyamaina skill**. Anni tables gurthupettukovadam avasaram ledu — **vetike vidhanam** telisthe chaalu. Moodu daarulu, easy nunchi kastam varaku.

Method 1 — F1 → Technical Information (easiest, use first).

Open the transaction in display mode, click the field, press F1, click Technical Information. Ten seconds, described fully in B5. **This answers 80% of "which table?" questions.**

Method 2 — SE84, search by name pattern.

If you know roughly what the table is about, search the dictionary:

SE84 → Dictionary Objects → Database Tables → enter **EK*** → Execute

This lists every table starting **EK** with its description — which is how you discover **EKBE** (PO history) exists without anyone telling you.

You can also search **DD02T** (table descriptions) in SE16N with a wildcard on the text:

SE16N → table **DD02T** → **DDTEXT** = *purchase order* → F8

That searches every table's description for a phrase. Slower, but exhaustive.

Method 3 — ST05, trace what the system actually reads.

The definitive method when the first two fail. **ST05 records every SQL statement** a transaction executes.

1. **/nST05** → **Activate Trace** (SQL trace)
2. In another session, run the transaction and do the thing you care about
3. Back in ST05 → **Deactivate Trace** → **Display Trace**
4. Read the list of tables SAP actually touched

This tells you the truth even when the screen field maps to a structure and F1 was unhelpful. It is what senior developers do when a value seems to come from nowhere.

A caution on ST05: it records everything, so keep the trace window to a few seconds, and never leave it running on a shared system. Trace your own session only.

Which to use when:

Do you have the field on a screen?

- Yes → F1 → Technical Information (10 seconds)
- No
 - Know roughly the name or topic?
 - SE84 / DD02T wildcard search (1 minute)
 - No idea at all
 - ST05 trace the transaction (5 minutes, definitive)

C4. Keys, MANDT and How Tables Join

Simple definition: every SAP table's first key field is the client, and every join between SAP tables includes it. Understanding key structure tells you what one row *means*.

Telugu: Prathi SAP table lo **modati key field MANDT** (client). ABAP lo nuvvu daanini **raayanavasaram ledu** — system automatic ga kalupthundi. Kaani key fields chusi, **oka row ante emiti** ani ardham chesukovali — adi oka purchase order aa, leka oka PO lo oka line aa?

Reading a key tells you the grain of the table:

Table	Key fields	One row is
EKKO	MANDT, EBELN	One purchase order
EKPO	MANDT, EBELN, EBELP	One line of one purchase order
EKET	MANDT, EBELN, EBELP, ETENR	One schedule line of one PO line
MARA	MANDT, MATNR	One material, globally
MARC	MANDT, MATNR, WERKS	One material in one plant
MARD	MANDT, MATNR, WERKS, LGORT	One material in one storage location

See the pattern: each extra key field makes the row more specific. **MARA** is the material everywhere; **MARC** is that material's data for one plant; **MARD** narrows to one storage location inside that plant. Choosing the wrong one is the most common cause of duplicated or missing rows in a report.

MANDT and your SELECT — the rule:

```
" You write this:
SELECT ebeln, buhrs, lifnr
  FROM ekko
 INTO TABLE @DATA(lt_po)
WHERE buhrs = '1000'.

" ABAP automatically executes this:
"   ... WHERE mandt = sy-mandt AND buhrs = '1000'
```

Never write `MANDT` in an Open SQL WHERE clause. The runtime adds it. Writing it yourself is flagged in code review, and in ABAP Cloud it is not permitted at all.

The standard header-item join:

```
SELECT k~ebeln,
       k~bukrs,
       k~lifnr,
       p~ebelp,
       p~matnr,
       p~menge,
       p~netpr
FROM   ekko AS k
INNER JOIN ekpo AS p
      ON k~ebeln = p~ebeln           "join on the header key only
 INTO TABLE @DATA(lt_items)
WHERE  k~bukrs = '1000'
      AND p~loekz = space.           "exclude deleted items
```

Two things to carry from that snippet: you join on the **header key**, and you almost always filter out **deletion-indicator** rows (`LOEKZ` in MM, `LOEVM` / `LVORM` elsewhere). Forgetting the deletion flag is why a report shows cancelled documents.

Part D — Materials Management

D1. MARA and MAKT — The Material Master

Simple definition: **MARA** holds one row per material for the whole company. **MAKT** holds its descriptions, one row per language.

Telugu: **MARA** — prathi material ki **oka row**, company motham ki. Idi SAP lo **anni kanna ekkuva vaade tables lo okati**. Description veru table lo — **MAKT** lo, prathi bhashaki oka row. Anduke material peru kavalante MAKt ni join cheyyali, inka **SPRAS** filter cheyyali.

Table	Holds	Key
MARA	Material master, general data	MANDT, MATNR
MAKT	Material descriptions	MANDT, MATNR, SPRAS

MARA — the fields you will use constantly:

Field	Meaning
MATNR	Material number — the key
MTART	Material type (FERT finished, ROH raw, HAWA trading goods)
MATKL	Material group
MEINS	Base unit of measure
BRGEW / NTGEW	Gross / net weight
GEWEI	Weight unit
LVORM	Deletion flag — filter it out
ERSDA / ERNAM	Created on / by

MAKT — three fields only: **MATNR**, **SPRAS** (language), **MAKTX** (the description text).

The join everyone writes:

```
SELECT a~matnr,
       a~mtart,
       a~matkl,
       a~meins,
       t~maktx
FROM   mara AS a
LEFT OUTER JOIN maktx AS t
  ON    t~matnr = a~matnr
       AND t~spras = @sy-langu    "language filter belongs in the ON clause
INTO TABLE @DATA(lt_mat)
WHERE  a~lvorm = space.
```

Two deliberate choices in that code, both worth understanding:

1. **LEFT OUTER JOIN**, so a material with no description in your language still appears — with a blank text rather than vanishing.
2. **The language filter is in the ON clause, not WHERE.** Putting **t~spras = sy-langu** in the WHERE clause silently turns a LEFT JOIN back into an INNER JOIN — the classic trap you met in the Phase 9 SQL notes, and it appears in ABAP exactly the same way.

The S/4HANA change to remember: **MATNR is now 40 characters** (it was 18 in ECC). Old code and old interfaces that assume 18 break. This is a standard interview question about S/4HANA differences.

D2. MARC, MARD, MBEW — Material Data by Level

Simple definition: the material master is stored in layers. Each table adds one level of organisational detail.

Telugu: Material data **layers lo** untundi. MARA = motham company ki. MARC = **oka plant** ki. MARD = **oka storage location** ki. MBEW = **valuation (dabbu)**. "Stock entha undi?" ani adigithe — adi **MARD** leda MARC, MARA kaadu. Ee teda teliyakapote report lo tappu vastundi.

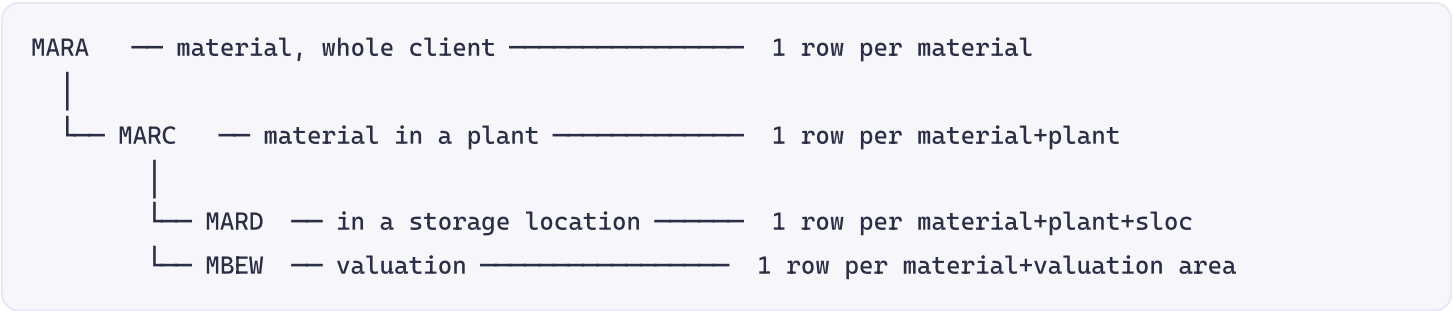


Table	Level	Key	Typical fields
MARC	Plant	MATNR, WERKS	DISPO (MRP controller), EKGRP (purchasing group), BESKZ (procurement type), LVORM
MARD	Storage location	MATNR, WERKS, LGORT	LABST (unrestricted stock), INSME (quality inspection), SPEME (blocked)
MBEW	Valuation area	MATNR, BWKEY , BWTAR	VERPR (moving average price), STPRS (standard price), PEINH (price unit), SALK3 (stock value)
MVKE	Sales org	MATNR, VKORG, VTWEG	Sales-specific settings

The question this answers: "how much stock do we have?" — the answer is **MARD-LABST**, summed across storage locations, for a given material and plant. Not MARA, which has no stock at all.

A caution on prices: **MBEW-VERPR** and **STPRS** must be divided by **PEINH** (price unit) to get the price per single unit. A material priced "500 per 100 pieces" stores **VERPR = 500**, **PEINH = 100**. Forgetting this makes valuations wrong by a factor of a hundred — a genuine production bug people ship.

D3. EBAN — Purchase Requisitions

Simple definition: **EBAN** holds purchase requisitions — the internal "please buy this" request that comes *before* a purchase order.

Telugu: **EBAN** = purchase requisition. Idi **nee December portfolio app** yokka nija SAP table — nuvvu kattedi deeni chinna version. Requisition ante "idi konandi" ani lopala adigedi; adi approve ayyaka **purchase order** (EKKO/EKPO) avutundi.

Table	Holds	Key
EBAN	Purchase requisition items	MANDT, BANFN , BNFPO
EBKN	Requisition account assignment	MANDT, BANFN, BNFPO, ZEBKN

Note the unusual shape: EBAN has no separate header table — the requisition number **BANFN** and item **BNFPO** are both in one table. Requisitions are simple enough that SAP did not split them.

The fields that matter:

Field	Meaning
BANFN	Requisition number
BNFPO	Item number
MATNR	Material
WERKS	Plant
MENGE / MEINS	Quantity and unit
PREIS	Estimated price
LFDAT	Delivery date required
AFNAM	Requisitioner (who asked)
FRGKZ / FRGZU	Release (approval) indicator and status
EBELN / EBELP	The purchase order created from it — the link forward
LOEKZ	Deletion indicator

FRGKZ is the approval field — the real SAP equivalent of the status field in your portfolio app. Release strategies in SAP are configuration-driven and considerably more complex than a single status column, but conceptually it is the same idea: a requisition is blocked until approved.

The link to purchase orders: when a requisition is converted, **EBAN-EBELN** and **EBAN-EBELP** are filled with the resulting PO. So "which requisitions are still open?" is:

```
SELECT banfn, bnfpo, matnr, werks, menge, lfdat
FROM eban
INTO TABLE @DATA(lt_open_pr)
WHERE loekz = space          "not deleted
      AND ebeln = space.     "no PO created yet = still open
```

Why this table specifically matters to you: your portfolio app models this process. When an interviewer asks what you built, saying "*a purchase requisition approval app — the same process SAP handles in EBAN with release strategies*" shows you know the real object your toy is modelled on.

D4. EKKO, EKPO, EKET, EKBE — Purchase Orders

Simple definition: the purchase order tables — the most-used document tables in MM, and the ones most likely to appear in an interview question.

Telugu: Purchase order tables. **EKKO** = header (oka PO), **EKPO** = items (PO lo prathi line), **EKET** = schedule lines (eppudu delivery), **EKBE** = history (goods receipt, invoice jariginda). Ee **naalugu perlu** gattiga gurthupettuko — MM lo prathi prasnaki ivi kavali.

Table	Holds	Key	One row is
EKKO	PO header	MANDT, EBELN	One purchase order
EKPO	PO items	MANDT, EBELN, EBELP	One line of a PO
EKET	Schedule lines	MANDT, EBELN, EBELP, ETENR	One delivery date for a line
EKBE	PO history	MANDT, EBELN, EBELP, ZEKKN, VGABE, GJAHR, BELNR	One goods receipt or invoice against a line
EKKN	Account assignment	MANDT, EBELN, EBELP, ZEKKN	Cost centre / order for a line

EKKO — header fields:

Field	Meaning
EBELN	PO number
BUKRS	Company code
BSART	Document type (NB standard, UB stock transport...)
LIFNR	Vendor
EKORG / EKGRP	Purchasing organisation / group
WAERS	Currency
AEDAT	Created on
FRGKE / FRGZU	Release status

EKPO — item fields:

Field	Meaning
EBELP	Item number

Field	Meaning
MATNR	Material
WERKS / LGORT	Plant / storage location
MENGE / MEINS	Quantity and unit
NETPR / PEINH	Net price and price unit
NETWR	Net value of the line
LOEKZ	Deletion indicator — always filter
ELIKZ / EREKZ	Delivery completed / invoice completed

EKET — the schedule lines, which is where the delivery date actually lives:

Field	Meaning
ETENR	Schedule line number
EINDT	Delivery date — the field from the B5 example
MENGE	Scheduled quantity
WEMNG	Quantity already received

This is the detail people get wrong: the delivery date is **not** on the PO item. It is on the schedule line, because one item can have several delivery dates. **EKET-EINDT** — that is why the F1 trick in B5 returned **EKET** , not **EKPO** .

EKBE — the history, which answers "has this actually been delivered or paid?":

VGABE value	Means
1	Goods receipt
2	Invoice receipt
3	Subsequent debit/credit

A useful three-table query — open PO lines with their delivery dates:

```

SELECT k~ebeln, k~lifnr, k~bukrs,
       p~ebelp, p~matnr, p~menge, p~netwr,
       t~eindt
FROM ekko AS k
INNER JOIN ekpo AS p ON k~ebeln = p~ebeln
INNER JOIN eket AS t ON p~ebeln = t~ebeln
                   AND p~ebelp = t~ebelp
INTO TABLE @DATA(lt_open)
WHERE k~bukrs = '1000'
      AND p~loekz = space           "not deleted"
      AND p~elikz = space.         "delivery not yet complete"

```

Say this out loud until it is natural: "EKKO header, EKPO item, EKET schedule line, EKBE history." It is the single most likely table question in an MM-flavoured interview.

Part E — Sales and Distribution

E1. VBAK, VBAP, VBEP — Sales Orders

Simple definition: the sales order tables. Same header/item shape as purchasing, mirrored on the selling side.

Telugu: Sales order tables. **VBAK** = header, **VBAP** = items, **VBEP** = schedule lines. Purchasing lo EKKO/EKPO/EKET laage — **ade structure**, kaani ammakam vaipu. Oka pattern nerchukunte, rendo di sulabham.

Table	Holds	Key
VBAK	Sales document header	MANDT, VBELN
VBAP	Sales document items	MANDT, VBELN, POSNR
VBEP	Schedule lines	MANDT, VBELN, POSNR, ETENR
VBPA	Partners (sold-to, ship-to, bill-to)	MANDT, VBELN, POSNR, PARVW
VBKD	Business data (payment terms, Incoterms)	MANDT, VBELN, POSNR

VBAK — header fields:

Field	Meaning
VBELN	Sales document number
AUART	Document type (OR standard order, QT quotation, RE returns)
KUNNR	Sold-to customer

Field	Meaning
VKORG / VTWEG / SPART	Sales org / distribution channel / division
NETWR	Net value
WAERK	Currency
ERDAT / ERNAM	Created on / by

VBAP — item fields:

Field	Meaning
POSNR	Item number
MATNR	Material
KWMENG	Order quantity
NETWR	Net value of the item
WERKS	Delivering plant
ABGRU	Rejection reason — if filled, the item is cancelled

Two S/4HANA changes that come up in interviews:

Change	Detail
VBUK and VBUP are gone	Status was stored in separate header/item status tables in ECC. In S/4HANA those tables were removed and the status fields moved into VBAK and VBAP directly . Old code selecting from VBUK breaks.
KONV → PRCD_ELEMENTS	The pricing conditions table was renamed. Old code referencing KONV must be adjusted.

Filter rejected items. `VBAP-ABGRU <> space` means the line was rejected. Reports that forget this show cancelled business as live — the SD equivalent of forgetting `LOEKZ`.

E2. LIKP and LIPS — Deliveries

Simple definition: the delivery documents — created from a sales order when goods are actually picked and shipped.

Telugu: Sales order create ayyaka, vastuvulu pampinchadaniki **delivery** document tayaravutundi. **LIKP** = header, **LIPS** = items. Ee rendu **warehouse** panilo mukhyam.

Table	Holds	Key
LIKP	Delivery header	MANDT, VBELN

Table	Holds	Key
LIPS	Delivery items	MANDT, VBELN, POSNR

LIKP fields: **VBELN** delivery number, **LFART** delivery type, **KUNNR** ship-to, **WADAT** planned goods-issue date, **WADAT_IST** actual goods-issue date, **VSTEL** shipping point, **LFDAT** delivery date.

LIPS fields: **POSNR** item, **MATNR** material, **LFIMG** delivery quantity, **WERKS** plant, **LGORT** storage location, **VBEL** / **VGPOS** — the **preceding document**, i.e. the sales order and item this line came from.

VBEL is how documents chain together. Every follow-on document stores the number of the document it came from. Delivery → sales order, billing → delivery, and so on. It is the manual way to trace a chain, and it is why the next table exists.

Note **WADAT_IST** : blank means goods issue has not been posted — the delivery is created but has not actually left. "Which deliveries are still not shipped?" is **WADAT_IST = space** .

E3. VBRK, VBRP and VBFA — Billing and Document Flow

Simple definition: billing documents (invoices), plus **VBFA** — the table that links every document in a process chain to the next.

Telugu: **VBRK/VBRP** = invoice header/items. Kaani ee section lo **asalu mukhyamainadi VBFA** — adi "ee order nunchi e delivery vachindi, daani nunchi e invoice vachindi" ani **motham gollusu (chain)** ni chupistundi. Idi SD lo chala shakti vantidi.

Table	Holds	Key
VBRK	Billing header	MANDT, VBELN
VBRP	Billing items	MANDT, VBELN, POSNR
VBFA	Document flow	MANDT, VBELV, VBELN, POSNV, POSNN, VBTYP_N

VBRK fields: **VBELN** invoice number, **FKART** billing type, **KUNRG** payer, **NETWR** net value, **WAERK** currency, **FKDAT** billing date, **RFBSK** posting status to accounting, **BELNR** the resulting accounting document.

VBFA — the document flow table, and how to read it:

Field	Meaning
VBELV	Preceding document number
POSNV	Preceding item
VBELN	Subsequent document number
POSNN	Subsequent item
VBTYP_N	Category of the subsequent document

Common **VBTP** values:

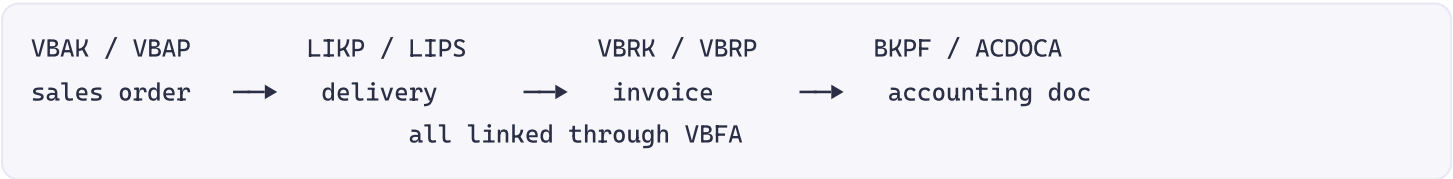
Value	Document
C	Sales order
J	Delivery
M	Invoice
H	Returns
R	Goods movement

What VBFA gives you: ask "what happened to sales order 12345?" and one select answers it:

```
SELECT vbelv, posnv, vbeln, posnn, vbtyp_n, rfmng
FROM vbfa
INTO TABLE @DATA(lt_flow)
WHERE vbelv = '0000012345'.
```

You get the deliveries, the goods issues and the invoices that followed — the whole chain, without joining five tables by hand.

The order-to-cash chain, in tables:



That diagram is worth memorising — it is the single most common "explain a business process technically" question in an SD interview, and it maps directly onto the Order-to-Cash process described in Part C of your Master Notes.

Part F — Finance

F1. BKPF and BSEG — The Classic Accounting Document

Simple definition: every financial posting in SAP creates an accounting document: one header in **BKPF** and several line items in **BSEG**.

Telugu: SAP lo **prathi dabbu leka-deka (posting)** oka accounting document tayararu chestundi. **BKPF** = header, **BSEG** = lines. Prathi document lo debit inka credit samanam ga undali. Kaani **S/4HANA lo idi maarindi** — tarvaata topic chudu.

Table	Holds	Key
BKPF	Accounting document header	MANDT, BUKRS , BELNR , GJAHR
BSEG	Accounting document line items	MANDT, BUKRS, BELNR, GJAHR, BUZEI

Note the three-part key. A financial document is identified by **company code + document number + fiscal year** — not by document number alone. Document numbers restart each year, so `BELNR = 100000001` is ambiguous without `GJAHR`.

BKPF fields: `BLART` document type, `BLDAT` document date, `BUDAT` **posting date**, `WAERS` currency, `XBLNR` reference, `USNAM` who posted, `TCODE` which transaction created it, `STBLG` reversal document.

BSEG fields: `BUZEI` line number, `SHKZG` debit/credit indicator (**S** = debit, **H** = credit), `DMBTR` amount in local currency, `WRBTR` amount in document currency, `HKONT` G/L account, `KOSTL` cost centre, `LIFNR` / `KUNNR` vendor/customer, `MATNR` , `ZUONR` assignment.

The important technical fact about BSEG in ECC: it was a **cluster table** — several logical tables physically stored together, compressed. That meant you **could not** select on non-key fields efficiently, which is why SAP maintained separate index tables (`BSID` , `BSAD` , `BSIK` , `BSAK` , `BSIS` , `BSAS`) just to make searching possible.

F2. ACDOCA — The Universal Journal

Simple definition: in S/4HANA, SAP replaced that whole tangle with **one** line-item table: **ACDOCA**, the Universal Journal.

Telugu: Idi **S/4HANA lo anni kanna pedda marpu**. Mundu FI, CO, asset accounting — anniki vere vere tables, inka index tables, totals tables. Ippudu **oke table — ACDOCA**. Interviews lo "S/4HANA lo em maarindi?" ani adigithe, **modata idi cheppu**.

What ACDOCA replaced:

In ECC	In S/4HANA
BSEG (line items)	ACDOCA
BSID / BSAD / BSIK / BSAK / BSIS / BSAS (index tables)	Removed — replaced by CDS views over ACDOCA
GLT0, FAGLFLEXT (totals)	Removed — totals computed on the fly by HANA
COEP (controlling line items)	Merged into ACDOCA
ANEP / ANLC (asset accounting)	Merged into ACDOCA
Separate FI and CO documents	One document, one table

Why this was possible: the index and totals tables existed only because a disk-based database could not aggregate millions of rows quickly enough. HANA can. So SAP deleted the redundancy and kept one wide table — the code-to-data principle applied at the largest possible scale.

What it means practically:

- **FI and CO are reconciled by construction.** They are the same row. The endless FI-vs-CO reconciliation work of the ECC era disappears.
- **BKPF still exists** as the document header.
- **BSEG still exists** for compatibility, but ACDOCA is the source of truth.
- The old index tables are provided as **compatibility views**, so legacy code still runs — but new code should never use them.

The interview answer, ready to say: "S/4HANA introduced the Universal Journal, ACDOCA. It merges FI, CO, asset accounting and material ledger line items into a single table, and removes the index and totals tables because HANA can aggregate on the fly. FI-CO reconciliation is no longer needed because they are literally the same record."

That answer, said fluently, is worth more than a dozen memorised table names.

F3. SKA1, SKB1 and T001 — Accounts and Company Codes

Simple definition: the master data behind financial postings — the chart of accounts and the legal entities.

Telugu: Postings ki venaka unna master data. **T001** = company codes (legal entities). **SKA1/SKB1** = G/L accounts. Ee tables chinnavi kaani prathi finance report lo join avutayi.

Table	Holds	Key
T001	Company codes	MANDT, BUKRS
SKA1	G/L account master, chart of accounts level	MANDT, KTOPL , SAKNR
SKB1	G/L account master, company code level	MANDT, BUKRS , SAKNR
SKAT	G/L account descriptions	MANDT, SPRAS, KTOPL, SAKNR

Notice the two-level pattern again, exactly like MARA → MARC: **SKA1** defines the account globally in a chart of accounts; **SKB1** adds the settings for one specific company code. Same modelling idea repeating across modules — once you recognise it, new tables stop being surprising.

T001 fields worth knowing: **BUKRS** company code, **BUTXT** name, **ORT01** city, **LAND1** country, **WAERS** local currency, **KTOPL** chart of accounts, **PERIV** fiscal year variant.

Why you will join T001 constantly: any financial report showing more than one company code needs the company name and its currency, and both live here.

Part G — Partners, Inventory and Organisation

G1. KNA1, LFA1, BUT000 — Customers, Vendors, Business Partners

Simple definition: the master data for the people and companies you trade with — and the S/4HANA change that reorganised all of it.

Telugu: Customers inka vendors data. ECC lo customer (KNA1) inka vendor (LFA1) **vere vere** undevi. S/4HANA lo **Business Partner (BUT000)** ye central — oke company customer kuda vendor kuda ayite, **oke BP**. Kaani KNA1, LFA1 ippatiki kindu untayi. Idi interview question.

Table	Holds	Key
KNA1	Customer master, general	MANDT, KUNNR
KNB1	Customer, company code data	MANDT, KUNNR, BUKRS
KNVV	Customer, sales area data	MANDT, KUNNR, VKORG, VTWEG, SPART
LFA1	Vendor master, general	MANDT, LIFNR
LFB1	Vendor, company code data	MANDT, LIFNR, BUKRS
BUT000	Business Partner, general	MANDT, PARTNER
BUT020	BP addresses	MANDT, PARTNER, ADDRNUMBER
ADRC	Central address data	MANDT, ADDRNUMBER
CVI_CUST_LINK	Links BP ↔ customer	
CVI_VEND_LINK	Links BP ↔ vendor	

The S/4HANA Business Partner story, in three sentences: in ECC, a company that both sold to you and bought from you had two unconnected master records — a customer number and a vendor number. S/4HANA makes **Business Partner** the single mandatory entry point, created through transaction **BP**. The old tables still exist underneath and are kept in sync by **CVI** (Customer/Vendor Integration), so **KNA1** and **LFA1** remain valid to read.

Where addresses actually live — this catches people out. **KNA1** has address *fields*, but the authoritative address is in **ADRC**, reached via **KNA1-ADRNR** :

```
SELECT k~kunnr, k~name1,  
       a~street, a~city1, a~post_code1, a~country  
FROM kna1 AS k  
LEFT OUTER JOIN adrc AS a  
  ON a~addrnumber = k~adrnr  
INTO TABLE @DATA(lt_cust).
```

Interview-ready line: "In S/4HANA, Business Partner is mandatory — customers and vendors are created through BP, and CVI keeps KNA1 and LFA1 synchronised underneath. XD01 and FK01 are no longer the entry point."

G2. MKPF, MSEG and MATDOC — Stock Movements

Simple definition: every physical movement of stock — receipt, issue, transfer — creates a material document.

Telugu: Vastuvulu kadilinnappudu (vachinnappudu, velinnappudu, transfer ayinnappudu) — prathi sari oka **material document** tayaruvutundi. ECC lo **MKPF/MSEG**. S/4HANA lo **MATDOC** ane oke table — kaani MKPF/MSEG compatibility views la ippatiki pani chestayi.

Table	Holds	Key
MKPF	Material document header	MANDT, MBLNR , MJAHR
MSEG	Material document items	MANDT, MBLNR, MJAHR, ZEILE
MATDOC	S/4HANA: the single material document table	MANDT, MBLNR, MJAHR, ZEILE

The S/4HANA change, same pattern as ACDOCA: MKPF and MSEG were merged into **MATDOC**, along with the aggregate stock tables. **MKPF** and **MSEG** remain as **compatibility views** so old code still runs. New development should use MATDOC or, better, the released CDS views over it.

Key fields in MSEG/MATDOC:

Field	Meaning
BWART	Movement type — the code that says what kind of movement this is
MATNR / WERKS / LGORT	Material, plant, storage location
MENGE / MEINS	Quantity and unit
SHKZG	S = debit (stock in), H = credit (stock out)
DMBTR	Value in local currency
EBELN / EBELP	The purchase order, if it came from one
LIFNR / KUNNR	Vendor / customer

Movement types you will meet constantly:

BWART	Means
101	Goods receipt against a purchase order
102	Reversal of 101
201	Goods issue to a cost centre
261	Goods issue to a production order
301	Plant-to-plant transfer
601	Goods issue for a delivery (sales)

Why movement type matters to a developer: it is the single field that determines what a stock movement *means*. Nearly every inventory report filters on **BWART**. Recognising 101 and 601 in particular marks you out as someone who has looked at real data.

G3. The Organisational Tables

Simple definition: the small configuration tables defining the company's structure. You join them constantly to turn codes into names.

Telugu: Ivi chinna configuration tables — company nirmanam ni cheptayi. Nee report lo "1000" ani plant code kanapadithe, aa **peru** kavali — adi T001W lo undi. Anduke ee tables ni prathi report lo join chestavu.

Table	Holds	Key	Name field
T001	Company codes	BUKRS	BUTXT
T001W	Plants	WERKS	NAME1
T001L	Storage locations	WERKS, LGORT	LGOBE
T024	Purchasing groups	EKGRP	EKNAM
T024E	Purchasing organisations	EKORG	EKOTX
TVKO	Sales organisations	VKORG	linked to T001
TVTW	Distribution channels	VTWEG	via TVTWT
TSPA	Divisions	SPART	via TSPAT
T880	Companies (group level)	RCOMP	NAME1

The pattern to notice: most of these have a **paired text table** ending in **T** (**TVTW** → **TVTWT** , **TSPA** → **TSPAT**) holding the description per language — exactly the pattern from C2. The ones that keep the name in the main table, like **T001W** , are the exceptions.

A worked example — turning codes into a readable report:

```
SELECT p~ebeln, p~ebel~p, p~matnr, p~menge,
       p~werks, w~name1 AS plant_name,
       k~bukrs, c~butxt AS company_name
FROM ekpo AS p
INNER JOIN ekko AS k ON k~ebeln = p~ebeln
LEFT OUTER JOIN t001w AS w ON w~werks = p~werks
LEFT OUTER JOIN t001 AS c ON c~bukrs = k~bukrs
INTO TABLE @DATA(lt_report)
WHERE p~loekz = space.
```

Why LEFT OUTER JOIN on the text tables: if a plant code is missing from **T001W** for any reason, an inner join would silently drop that purchase order line from your report. A left join keeps the row and shows a blank name.

For lookup and text tables, default to LEFT OUTER JOIN — it is the difference between a report that is wrong and a report that is merely incomplete, and only one of those gets noticed before go-live.

Part H — The Other Modules, Briefly

H1. Production, Quality, Maintenance and Warehouse

Simple definition: the tables outside MM/SD/FI. You will not use most of them, but recognising the names tells you which module a piece of code belongs to.

Telugu: Ivi MM, SD, FI kaakunda migilina modules. Nuvvu ivi **ippudu vaadavu** — kaani code lo ee perlu kanapadithe "idi e module di" ani telisi undali. Skim cheyyi, gurthupettukoku.

Production Planning (PP):

Table	Holds
AFKO	Production order header
AFPO	Production order item
AFVC	Order operations (the routing steps)
RESB	Reservations — the components an order needs
MAST	Links a material to its bill of materials
STKO / STPO	BOM header / BOM items
PLKO / PLPO	Routing header / routing operations
AUFG	Order master data — shared by production, maintenance, internal orders

Quality Management (QM): **QALS** inspection lot, **QAVE** usage decision, **QAMV** inspection characteristics.

Plant Maintenance (PM): **EQUI** equipment, **EQKT** equipment description, **IFLOT** functional location, **AFIH** maintenance order header (joins to **AUFG**).

Warehouse (classic WM): **LTAK** transfer order header, **LTAP** transfer order items, **LQUA** quantums (stock at bin level).

Extended Warehouse Management (EWM): everything lives in the **/SCWM/** namespace — **/SCWM/ORDIM_0** (warehouse tasks), **/SCWM/AQUA** (stock). If you see slashes in a table name, it is a namespaced component, and EWM is the most common one you will meet.

The one to notice: **AUFG**. It holds the master record for *any* kind of order — production, maintenance, internal. If code joins to **AUFG**, it is working with orders of some sort, and **AUFG-AUART** tells you which kind.

H2. HR and the SuccessFactors Question

Simple definition: HR data in SAP is stored as **infotypes** — a completely different modelling pattern from everything else in this guide.

Telugu: HR data **vere vidhanga** untundi — **infotypes** ani antaru. Prathi infotype oka table, peru **PA + number** (PA0001, PA0002...). Inka prathi row ki **from-date, to-date** untayi — ante charitra (history) motham untundi. Chala companies ippudu HR ki **SuccessFactors** vaadutunnayi, SAP HR kaadu.

Table	Infotype	Holds
PA0000	0000	Actions — hiring, promotion, termination
PA0001	0001	Organisational assignment — company code, cost centre, position
PA0002	0002	Personal data — name, date of birth
PA0006	0006	Addresses
PA0008	0008	Basic pay
PA0105	0105	Communication — email, user ID
HRP1000	—	Organisational objects (positions, org units, jobs)
HRP1001	—	Relationships between those objects

Two things that make HR tables different:

1. **Every row is time-dependent.** **BEGDA** (from date) and **ENDDA** (to date) are part of the key. A person has *many* rows in PA0001 — one per period of their org assignment. To get today's data you filter **BEGDA <= sy-datum AND ENDDA >= sy-datum**. Forgetting this returns an employee's entire history instead of their current state.
2. **Authorization is much tighter.** HR data is personal data; access is restricted far more than logistics data, and reading it casually in a real system is a compliance issue, not a curiosity.

The 2026 context: most large customers now run **SuccessFactors** for HR rather than SAP HCM, with employee data replicated into S/4HANA for payroll and cost accounting. So HR tables matter mainly for **integration** work — see Part U of your Master Notes for how SuccessFactors and S/4HANA exchange data.

Practical advice: skim this section, do not study it. Unless you land specifically in an HCM role, **PA0001** and **PA0002** are the only two names you need to recognise.

Part I — System and Technical Tables

11. TSTC, TADIR and the DD* Tables

Simple definition: SAP stores its own metadata in tables — including a table listing every table. These are how you query the system about itself.

Telugu: SAP tana sonta **metadata** ni kuda tables lo ne daachutundi — **tables gurinchi cheppe table** kuda undi. Ivi telisthe, "ee system lo Z tables enni unnayi?", "ee field e tables lo vaadaru?" laanti prasnalaki **SELECT** tho jawaabu teesukovachu.

Table	Holds	Use it to
TSTC	Every transaction code	List or search T-codes
TSTCT	T-code descriptions (per language)	Search T-codes by description
TADIR	Directory of all repository objects	Find every object in a package, or by author
TDEVC	Packages	List packages
TRDIR	ABAP program directory	Find programs by attributes
DD02L / DD02T	Tables and their descriptions	Search tables by name or text
DD03L	Table fields	Find which tables contain a given field
DD04L / DD04T	Data elements	Search data elements
DD01L	Domains	
DD07L / DD07T	Domain fixed values and their texts	Decode a status code
T100	Message texts	Find where a message comes from
TFDIR	Function modules	
SEOCLASS	Global classes	
NRIV	Number range intervals	See how document numbers are assigned
TVARVC	Variant variables	Values used in scheduled report variants

Three queries that solve real problems:

"Which tables contain the field **MATNR**?"

```
SELECT tabname, fieldname, position
FROM dd03l
INTO TABLE @DATA(lt_tabs)
WHERE fieldname = 'MATNR'
AND tabname LIKE 'Z%'.      "restrict, or you get thousands"
```

"What Z tables exist in this system?"

```
SELECT l~tabname, t~ddtext
FROM dd02l AS l
LEFT OUTER JOIN dd02t AS t
ON t~tabname = l~tabname
AND t~ddlanguage = @sy-langu
INTO TABLE @DATA(lt_ztabs)
WHERE l~tabname LIKE 'Z%'
AND l~tabclass = 'TRANSP'.  "transparent tables only"
```


"Find a transaction by description" — via SE16N on `TSTCT` with `TTEXT = *purchase order*`. This is Method 2 from C3, applied to transactions instead of tables.

Why this section is worth more than it looks: being able to *query the dictionary* rather than browse it is a small thing that repeatedly saves an hour. It is also the sort of answer that makes an interviewer pause — most candidates would not think of selecting from `DD03L`.

12. Transport and User Tables

Simple definition: the tables behind transports and user administration — useful for answering "who changed this?" and "when did this go live?".

Telugu: Transports inka users venaka unna tables. "Idi evaru marchāru?", "ee code eppudu production ki vellindi?" ani telusukovadaniki ivi upayogam. Roju vaadavu, kaani telisthe manchidi.

Table	Holds
E070	Transport request header — status, owner, dates
E071	Objects inside each transport request
E07T	Transport request descriptions
USR02	User logon data — last login, lock status, validity
USR21 / ADRP	User name and personal details
AGR_DEFINE	Role definitions
AGR_USERS	Which users have which roles
AGR_TCODES	Which transactions a role permits
USR04 / UST04	User authorization profiles

Two questions these answer:

"Which transport contains my object?"

```
SELECT trkorr, object, obj_name
FROM e071
INTO TABLE @DATA(lt_tr)
WHERE obj_name = 'ZNV_CARRIER_LIST'.
```

"Who has access to transaction ME23N?" — join `AGR_TCODES` (roles containing the T-code) to `AGR_USERS` (users holding those roles). This is the query a security team runs, and knowing it exists is enough.

A note on `TADIR` versus `E071`: `TADIR` tells you where an object *lives* (its package and who created it). `E071` tells you which transports have *moved* it. Different questions — people confuse them.

Part J — Putting It to Work

J1. From a Screen Field to a Working SELECT

Simple definition: the complete worked example — the exact sequence you will repeat hundreds of times in your career.

Telugu: Idi **motham process** — oka udaharana tho. Nee career lo ee vidhanam ni **vandala sarlu** repeat chestavu. Ee oka topic ni gattiga practice cheyyi; migilina antha deeniki material matrame.

The request: "Give me a report of purchase orders for company code 1000 where the delivery date has passed but goods have not been received. Show the vendor name and the plant name."

Step 1 — find the delivery date field. You do not know where it lives. Open **ME23N** on any PO, click the delivery date, press **F1** → **Technical Information**.

Result: **Table** **EKET** , **field** **EINDT**

Step 2 — understand the table in SE11. **/nSE11** → **EKET** → Display → Fields tab.

Key: **MANDT** , **EBELN** , **EBELP** , **ETENR** . So one row is a **schedule line** of a PO item — not a PO. That tells you the grain, and it tells you the join path.

Step 3 — check real data in SE16N. **/nSE16N** → **EKET** → put a known PO number in **EBELN** → F8.

You see **EINDT** filled and **WEMNG** (quantity received) at zero for undelivered lines. That confirms which field means "not received".

Step 4 — find the surrounding tables. You need vendor (header) and plant (item), so: **EKKO** for **LIFNR** , **EKPO** for **WERKS** . Then names: **LFA1** for the vendor name, **T001W** for the plant name.

Step 5 — write the SELECT.

```
REPORT znv_overdue_po.

SELECT k~ebeln,
       k~lifnr,
       v~name1 AS vendor_name,
       p~ebelp,
       p~matnr,
       p~werks,
       w~name1 AS plant_name,
       t~eindt,
```

```

        t~menge,
        t~wemng
FROM ekko AS k
INNER JOIN ekpo AS p ON p~ebeln = k~ebeln
INNER JOIN eket AS t ON t~ebeln = p~ebeln
                AND t~ebelp = p~ebelp
LEFT OUTER JOIN lfa1 AS v ON v~lifnr = k~lifnr
LEFT OUTER JOIN t001w AS w ON w~werks = p~werks
INTO TABLE @DATA(lt_overdue)
WHERE k~bukrs = '1000'
      AND p~loekz = @space      "item not deleted
      AND p~elikz = @space      "delivery not marked complete
      AND t~eindt < @sy-datum    "delivery date has passed
      AND t~wemng = 0.          "nothing received yet

IF lt_overdue IS INITIAL.
  MESSAGE 'No overdue purchase orders found' TYPE 'S'.
  RETURN.
ENDIF.

LOOP AT lt_overdue INTO DATA(ls_row).
  WRITE: / ls_row-ebeln, ls_row-ebelp, ls_row-vendor_name,
          ls_row-plant_name, ls_row-eindt, ls_row-menge.
ENDLOOP.

```

Every decision in that statement, and why:

Decision	Reason
<code>INNER JOIN</code> for EKPO and EKET	A PO without items is meaningless — no row should survive
<code>LEFT OUTER JOIN</code> for LFA1 and T001W	A missing name must not delete the business row
<code>p~loekz = space</code>	Exclude deleted items
<code>p~elikz = space</code>	Exclude lines already flagged delivery-complete
<code>t~wemng = 0</code>	Nothing actually received
No <code>MANDT</code> anywhere	The runtime adds it
Named columns, not <code>SELECT *</code>	Fetch only what you need
<code>@DATA(...)</code> inline	Modern ABAP; the type is derived
Empty-result check	A report that says nothing is a report that looks broken

That five-step loop — F1, SE11, SE16N, find neighbours, write — is the job. Practise it until it is automatic and most ABAP report work becomes routine.

J2. Views and CDS Equivalents in S/4HANA

Simple definition: for most of the tables in this guide, S/4HANA also ships a **CDS view** that is easier and safer to use — and in ABAP Cloud, the view is the *only* thing you are allowed to use.

Telugu: Ee tables anni ippatiki unnayi. Kaani S/4HANA lo prathi dhaaniki oka **CDS view** kuda undi — adi vaadadam **manchidi**. Inka **ABAP Cloud lo** nuvvu tables ni direct ga touch cheyyalevu — **released CDS views** matrame vaadali. Ante nee December portfolio app lo, direct table access **panicheyadu**.

The CDS naming convention:

Prefix	Means
I_	Interface view — reusable, stable, the ones you consume
C_	Consumption view — shaped for one UI
P_	Private — SAP internal, do not use
E_	Extension view
A_	The OData API entity name

Common released views, mapped to the tables you now know:

Table you know	Released CDS view
MARA / MAKT	I_Product , I_ProductText
MARC	I_ProductPlant
EKKO	I_PurchaseOrder
EKPO	I_PurchaseOrderItem
EBAN	I_PurchaseRequisitionItem
VBAK	I_SalesOrder
VBAP	I_SalesOrderItem
KNA1 / LFA1 / BUT000	I_BusinessPartner
ACDOCA	I_JournalEntryItem
T001	I_CompanyCode
T001W	I_Plant

Why views are better than tables:

1. **Readable names.** `I_PurchaseOrder-PurchaseOrder` instead of `EKKO-EBELN`. Self-documenting code.
2. **Associations built in.** The view already knows how to reach the vendor and the items — no hand-written joins.
3. **Upgrade-stable.** SAP guarantees released views; underlying tables can change (as ACDOCA and MATDOC proved).
4. **Required in ABAP Cloud.** Direct access to unreleased tables simply does not compile.

How to check whether something is released: in ADT (Eclipse), open the object — the *API State* shows "Released for Cloud Development". You can also search the SAP Business Accelerator Hub.

So why learn the tables at all? Three reasons, all real:

- Every existing custom program at every client selects from tables directly. You must read that code.
- Debugging, SE16N and data analysis all work at the table level.
- Interviewers ask about tables, because that is what they learned.

The right stance for December: *"I know the underlying tables — EKKO, EKPO, EKET — and for new development I use the released CDS views like I_PurchaseOrder, because ABAP Cloud requires it and they are upgrade-stable."* That single sentence shows both the classical grounding and the modern practice, which is exactly the profile that is scarce.

J3. Practice Drills for Your 9–11 Slot

Simple definition: five short exercises. Each takes 15–20 minutes and builds one specific reflex.

Telugu: Aidu chinna drills. Prathi okkati 15-20 nimishalu. Ee drills **nee 9–11 slot lo** cheyyi — chadavadam kaadu, **chesi chudadam**. Vaaraniki okati chaalu.

Drill 1 — the F1 hunt (do this first, repeat weekly). Open `ME23N` on any purchase order. Press F1 → Technical Information on ten different fields. Write down each table and field. You should end with `EKKO`, `EKPO`, `EKET` and at least one structure. Repeat next week with `VA03`, then `MM03`.

Drill 2 — grain check. Open `MARA`, `MARC`, `MARD` in SE11. For each, write one sentence: *"one row in this table is ..."*. Then open the same material in SE16N in all three and count the rows. Seeing one row in MARA and six in MARD makes the layering permanent.

Drill 3 — the header-item join. Write a report joining `EKKO` and `EKPO`, filtered on one company code, excluding deleted items. Run it. Then break it deliberately: remove the `LOEKZ` filter and see how many extra rows appear. Then swap the INNER JOIN for a LEFT OUTER and observe what changes.

Drill 4 — find a table blind. Pick a field you have never looked up — the payment terms on a purchase order, or the incoterms on a sales order. Find its table **without asking anyone**, using the C3 methods in order. Time yourself. Under two minutes is the target.

Drill 5 — query the dictionary. Write the `DD03L` query from I1 to find every table containing `LIFNR`. Then use `DD02T` to find every table whose description mentions "delivery". This makes the system feel searchable rather than infinite.

How to fit them in: one drill per week, inside the existing 9–11 structure — they belong in the "main task" block. Five weeks covers all five, and by October the reflexes are there before the CDS and RAP work starts.

J4. Twenty Questions You Should Be Able to Answer

Simple definition: the T-code and table questions that recur in entry-level SAP interviews, with the sharp answer to each.

Telugu: Ee **iravai prasnalu** interviews lo malli malli vastayi. Prathi dhaaniki **oka chinna vaakyam** lo jawaabu cheppagalagali. November lo ivi roju revise cheyyi.

#	Question	Answer
1	What is a transaction code?	A short name that starts a program or screen; defined in SE93, stored in TSTC
2	Difference between SE11 and SE16N?	SE11 shows a table's structure ; SE16N shows its contents
3	How do you find the table behind a screen field?	F1 → Technical Information
4	Purchase order tables?	EKKO header, EKPO item, EKET schedule lines, EKBE history
5	Where is the PO delivery date?	EKET-EINDT — on the schedule line, not the item
6	Sales order tables?	VBAK header, VBAP item, VBEP schedule lines
7	Material master tables?	MARA general, MARC plant, MARD storage location, MBEW valuation, MAKT descriptions
8	Where is stock quantity?	MARD-LABST for unrestricted stock
9	Customer and vendor tables?	KNA1 customer, LFA1 vendor; in S/4HANA BUT000 via Business Partner and CVI
10	Accounting document tables?	BKPF header, BSEG items; in S/4HANA ACDOCA is the Universal Journal
11	What is ACDOCA?	The Universal Journal — merges FI, CO, asset accounting into one line-item table; removes index and totals tables
12	What replaced MKPF/MSEG?	MATDOC — the old names remain as compatibility views
13	What happened to VBUK/VBUP?	Removed in S/4HANA; status fields moved into VBAK and VBAP
14	Why is MANDT the first key field?	It makes data client-specific; the runtime adds it to every Open SQL WHERE clause automatically
15	How do you trace a sales order to its invoice?	VBFA , the document flow table
16	Where do descriptions live?	Separate text tables ending in T, keyed by language — MAKT, TSTCT, DD02T

#	Question	Answer
17	How do you find which tables use a field?	Query DD03L , or use the where-used list on the data element
18	How do you find a table when F1 gives a structure?	Take the data element, use where-used on table fields — or trace with ST05
19	Should you use tables or CDS views in new code?	Released CDS views — required in ABAP Cloud, upgrade-stable, with built-in associations
20	Which T-code shows short dumps?	ST22

How to practise: cover the right column, answer out loud, ten minutes a day. Do this in November alongside the twenty terms in Part T of your Master Notes. Between the two lists you will have forty ready answers — which is more preparation than most candidates arrive with.

One last thing. Nobody expects you to know every table. They expect you to know **the common ones** and to have a **method** for the rest. If a question stumps you, the strongest answer is not silence and not a guess — it is: *"I don't know that one offhand, but I'd find it with F1 → Technical Information on the screen field, or trace it with ST05."* That answer is correct, it is honest, and it tells the interviewer exactly what they wanted to know.